A path is a sequence of alternating vertices and edges that starts at a vertex and ends at a vertex such that each edge is incident to its predecessor and successor vertex. A cycle is a path that starts and ends at the same vertex, and that includes at least one edge. We say that a path is simple if each vertex in the path is distinct, and we say that a cycle is simple if each vertex in the cycle is distinct, except for the ·rst and last one. A directed path is a path such that all edges are directed and are traversed along their direction. A directed cycle is similarly de·ned. For example, in Figure 14.2, (BOS, NW 35, JFK, AA 1387, DFW) is a directed simple path, and (LAX, UA 120, ORD, UA 877, DFW, AA 49, LAX) is a directed simple cycle. Note that a directed graph may have a cycle consisting of two edges with opposite direction between the same pair of vertices, for example (ORD, UA 877, DFW, DL 335, ORD) in Figure 14.2. A directed graph is acyclic if it has no directed cycles. For example, if we were to remove the edge UA 877 from the graph in Figure 14.2, the remaining graph is acyclic. If a graph is simple, we may omit the edges when describing path P or cycle C, as these are well de·ned, in which case P is a list of adjacent vertices and C is a cycle of adjacent vertices.

Example 14.6: Given a graph G representing a city map (see Example 14.3), we can model a couple driving to dinner at a recommended restaurant as traversing a path though G. If they know the way, and do not accidentally go through the same intersection twice, then they traverse a simple path in G. Likewise, we can model the entire trip the couple takes, from their home to the restaurant and back, as a cycle. If they go home from the restaurant in a completely different way than how they went, not even going through the same intersection twice, then their entire round trip is a simple cycle. Finally, if they travel along one-way streets for their entire trip, we can model their night out as a directed cycle.

Given vertices u and v of a (directed) graph G, we say that u reaches v, and that v is reachable from u, if G has a (directed) path from u to v. In an undirected graph, the notion of reachability is symmetric, that is to say, u reaches v if an only if v reaches u. However, in a directed graph, it is possible that u reaches v but v does not reach u, because a directed path must be traversed according to the respective directions of the edges. A graph is connected if, for any two vertices, there is a path between them. A directed graph ·
G is strongly connected if for any two vertices u
and v of ·
G, u reaches v and v reaches u. (See Figure 14.3 for some examples.)

A subgraph of a graph G is a graph H whose vertices and edges are subsets of the vertices and edges of G, respectively. A spanning subgraph of G is a subgraph of G that contains all the vertices of the graph G. If a graph G is not connected, its maximal connected subgraphs are called the connected components of G. A forest is a graph without cycles. A tree is a connected forest, that is, a connected graph without cycles. A spanning tree of a graph is a spanning subgraph that is a tree. (Note that this de·nition of a tree is somewhat different from the one given in Chapter 8, as there is not necessarily a designated root.)

Example 14.2: We can associate with an object-oriented program a graph whose vertices represent the classes de·ned in the program, and whose edges indicate inheritance between classes. There is an edge from a vertex v to a vertex u if the class for v inherits from the class for u. Such edges are directed because the inheritance relation only goes in one direction (that is, it is asymmetric).

If all the edges in a graph are undirected, then we say the graph is an undirected graph. Likewise, a directed graph, also called a digraph, is a graph whose edges are all directed. A graph that has both directed and undirected edges is often called a mixed graph. Note that an undirected or mixed graph can be converted into a directed graph by replacing every undirected edge (u,v) by the pair of directed edges (u,v) and (v,u). It is often useful, however, to keep undirected and mixed graphs represented as they are, for such graphs have several applications, as in the following example.

Example 14.3: A city map can be modeled as a graph whose vertices are intersections or dead ends, and whose edges are stretches of streets without intersections. This graph has both undirected edges, which correspond to stretches of two-way streets, and directed edges, which correspond to stretches of one-way streets. Thus, in this way, a graph modeling a city map is a mixed graph.

Example 14.4: Physical examples of graphs are present in the electrical wiring and plumbing networks of a building. Such networks can be modeled as graphs, where each connector, ·xture, or outlet is viewed as a vertex, and each uninterrupted stretch of wire or pipe is viewed as an edge. Such graphs are actually components of much larger graphs, namely the local power and water distribution networks. Depending on the speci·c aspects of these graphs that we are interested in, we may consider their edges as undirected or directed, for, in principle, water can ·ow in a pipe and current can ·ow in a wire in either direction.

The two vertices joined by an edge are called the end vertices (or endpoints) of the edge. If an edge is directed, its ·rst endpoint is its origin and the other is the destination of the edge. Two vertices u and v are said to be adjacent if there is an edge whose end vertices are u and v. An edge is said to be incident to a vertex if the vertex is one of the edge·s endpoints. The outgoing edges of a vertex are the directed edges whose origin is that vertex. The incoming edges of a vertex are the directed edges whose destination is that vertex. The degree of a vertex v, denoted deg(v), is the number of incident edges of v. The in-degree and out-degree of a vertex v are the number of the incoming and outgoing edges of v, and are denoted indeg(v) and outdeg(v), respectively.

14.1.1 The Graph ADT

A graph is a collection of vertices and edges. We model the abstraction as a combination of three data types: Vertex, Edge, and Graph. A Vertex is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code); we assume it supports a method, element(), to retrieve the stored element. An Edge also stores an associated object (e.g., a ·ight number, travel distance, cost), retrieved with the element() method. In addition, we assume that an Edge supports the following methods:

endpoints(): Return a tuple (u,v) such that vertex u is the origin of the edge and vertex v is the destination; for an undirected graph, the orientation is arbitrary.

opposite(v): Assuming vertex v is one endpoint of the edge (either origin or destination), return the other endpoint.

The primary abstraction for a graph is the Graph ADT. We presume that a graph can be either undirected or directed, with the designation declared upon construction; recall that a mixed graph can be represented as a directed graph, modeling edge {u,v} as a pair of directed edges (u,v) and (v,u). The Graph ADT includes the following methods:

vertex count(): Return the number of vertices of the graph.

vertices(): Return an iteration of all the vertices of the graph.

edge count(): Return the number of edges of the graph.

edges(): Return an iteration of all the edges of the graph.

get edge(u,v): Return the edge from vertex u to vertex v, if one exists; otherwise return None. For an undirected graph, there is no difference between get edge(u,v) and get edge(v,u).

degree(v, out=True): For an undirected graph, return the number of edges incident to vertex v. For a directed graph, return the number of outgoing (resp. incoming) edges incident to vertex v, as designated by the optional parameter.

incident edges(v, out=True): Return an iteration of all edges incident to vertex v. In the case of a directed graph, report outgoing edges by default; report incoming edges if the optional parameter is set to False.

insert vertex(x=None): Create and return a new Vertex storing element x.

insert edge(u, v, x=None): Create and return a new Edge from vertex u to vertex v, storing element x (None by default).

remove vertex(v): Remove vertex v and all its incident edges from the graph.

remove edge(e): Remove edge e from the graph.

Running Time of Depth-First Search
In terms of its running time, depth-·rst search is an ef·cient method for traversing a graph. Note that DFS is called at most once on each vertex (since it gets marked as visited), and therefore every edge is examined at most twice for an undirected graph, once from each of its end vertices, and at most once in a directed graph, from its origin vertex. If we let ns ·n be the number of vertices reachable from a vertex s, and ms ·m be the number of incident edges to those vertices, a DFS starting at s runs in O(ns+ms) time, provided the following conditions are satis·ed:
· The graph is represented by a data structure such that creating and iterating through the incident edges(v) takes O(deg(v)) time, and the e.opposite(v) method takes O(1) time. The adjacency list structure is one such structure, but the adjacency matrix structure is not.
· We have a way to ·mark· a vertex or edge as explored, and to test if a vertex or edge has been explored in O(1) time. We discuss ways of implementing DFS to achieve this goal in the next section.
Given the assumptions above, we can solve a number of interesting problems.
Proposition 14.14: Let G be an undirected graph with n vertices and m edges. A DFS traversal of G can be performed in O(n + m) time, and can be used to solve the following problems in O(n+m) time:
· Computing a path between two given vertices of G, if one exists.
· Testing whether G is connected.
· Computing a spanning tree of G, if G is connected.
· Computing the connected components of G.
· Computing a cycle in G, or reporting that G has no cycles.
Proposition 14.15: Let ·
G be a directed graph with n vertices and m edges. A
DFS traversal of ·
G can be performed in O(n + m) time, and can be used to solve
the following problems in O(n+m) time:
· Computing a directed path between two given vertices of ·
G, if one exists.
· Computing the set of vertices of ·
G that are reachable from a given vertex s.
· Testing whether ·
G is strongly connected.
· Computing a directed cycle in ·
G, or reporting that ·
G is acyclic.
· Computing the transitive closure of ·
G (see Section 14.4).
The justi·cation of Propositions 14.14 and 14.15 is based on algorithms that use slightly modi·ed versions of the DFS algorithm as subroutines. We will explore some of those extensions in the remainder of this section.

14.4
Transitive Closure
We have seen that graph traversals can be used to answer basic questions of reachability in a directed graph. In particular, if we are interested in knowing whether there is a path from vertex u to vertex v in a graph, we can perform a DFS or BFS traversal starting at u and observe whether v is discovered. If representing a graph with an adjacency list or adjacency map, we can answer the question of reachability for u and v in O(n+m) time (see Propositions 14.15 and 14.17).
In certain applications, we may wish to answer many reachability queries more ef·ciently, in which case it may be worthwhile to precompute a more convenient representation of a graph. For example, the ·rst step for a service that computes driving directions from an origin to a destination might be to assess whether the destination is reachable. Similarly, in an electricity network, we may wish to be able to quickly determine whether current ·ows from one particular vertex to another. Motivated by such applications, we introduce the following de·nition. The transitive closure of a directed graph ·
G is itself a directed graph ·
G·such that the
vertices of ·
G·are the same as the vertices of ·
G, and ·
G·has an edge (u,v), whenever ·
G has a directed path from u to v (including the case where (u,v) is an edge of the original ·
G).
If a graph is represented as an adjacency list or adjacency map, we can compute its transitive closure in O(n(n+m)) time by making use of n graph traversals, one from each starting vertex. For example, a DFS starting at vertex u can be used to determine all vertices reachable from u, and thus a collection of edges originating with u in the transitive closure.
In the remainder of this section, we explore an alternative technique for computing the transitive closure of a directed graph that is particularly well suited for when a directed graph is represented by a data structure that supports O(1)-time lookup for the get edge(u,v) method (for example, the adjacency-matrix structure). Let ·
G
be a directed graph with n vertices and m edges. We compute the transitive closure of ·
G in a series of rounds. We initialize ·
G0 = ·
G. We also arbitrarily number the
vertices of ·
G as v1,v2,..., vn. We then begin the computation of the rounds, beginning with round 1. In a generic round k, we construct directed graph ·
Gk starting
with ·
Gk = ·

14.1 Graphs

A graph is a way of representing relationships that exist between pairs of objects. That is, a graph is a set of objects, called vertices, together with a collection of pairwise connections between them, called edges. Graphs have applications in modeling many domains, including mapping, transportation, computer networks, and electrical engineering. By the way, this notion of a ·graph· should not be confused with bar charts and function plots, as these kinds of ·graphs· are unrelated to the topic of this chapter.

Viewed abstractly, a graph G is simply a set V of vertices and a collection E of pairs of vertices from V, called edges. Thus, a graph is a way of representing connections or relationships between pairs of objects from some setV. Incidentally, some books use different terminology for graphs and refer to what we call vertices as nodes and what we call edges as arcs. We use the terms ·vertices· and ·edges.·

Edges in a graph are either directed or undirected. An edge (u,v) is said to be directed from u to v if the pair (u,v) is ordered, with u preceding v. An edge (u,v) is said to be undirected if the pair (u,v) is not ordered. Undirected edges are sometimes denoted with set notation, as {u,v}, but for simplicity we use the pair notation (u,v), noting that in the undirected case (u,v) is the same as (v,u). Graphs are typically visualized by drawing the vertices as ovals or rectangles and the edges as segments or curves connecting pairs of ovals and rectangles. The following are some examples of directed and undirected graphs.

Example 14.1: We can visualize collaborations among the researchers of a certain discipline by constructing a graph whose vertices are associated with the researchers themselves, and whose edges connect pairs of vertices associated with researchers who have coauthored a paper or book. (See Figure 14.1.) Such edges are undirected because coauthorship is a symmetric relation; that is, if A has coauthored something with B, then B necessarily has coauthored something with A.

Chiang
Goldwasser
Tamassia
Goodrich
Garg
Snoeyink
Tollis
Vitter
Preparata

Figure 14.1: Graph of coauthorship among some authors.

1
class Graph:
2
⋯Representation of a simple graph using an adjacency map.⋯
3
4
def
init
(self, directed=False):
5
⋯Create an empty graph (undirected, by default).
6
7
Graph is directed if optional paramter is set to True.
8
⋯
9
self. outgoing = { }
10
only create second map for directed graph; use alias for undirected
11
self. incoming = { } if directed else self. outgoing
12
13
def is directed(self):
14
⋯Return True if this is a directed graph; False if undirected.
15
16
Property is based on the original declaration of the graph, not its contents.
17
⋯
18
return self. incoming is not self. outgoing # directed if maps are distinct
19
20
def vertex count(self):
21
⋯Return the number of vertices in the graph.⋯
22
return len(self. outgoing)
23
24
def vertices(self):
25
⋯Return an iteration of all vertices of the graph.⋯
26

For directed graph, ·
G, we may wish to test whether it is strongly connected, that
is, whether for every pair of vertices u and v, both u reaches v and v reaches u. If we start an independent call to DFS from each vertex, we could determine whether this was the case, but those n calls when combined would run in O(n(n+m)). However, we can determine if ·
G is strongly connected much faster than this, requiring only
two depth-·rst searches.
We begin by performing a depth-·rst search of our directed graph ·
G starting at
an arbitrary vertex s. If there is any vertex of ·
G that is not visited by this traversal,
and is not reachable from s, then the graph is not strongly connected. If this ·rst depth-·rst search visits each vertex of ·
G, we need to then check whether s is reach-
able from all other vertices. Conceptually, we can accomplish this by making a copy of graph ·
G, but with the orientation of all edges reversed. A depth-·rst search
starting at s in the reversed graph will reach every vertex that could reach s in the original. In practice, a better approach than making a new graph is to reimplement a version of the DFS method that loops through all incoming edges to the current vertex, rather than all outgoing edges. Since this algorithm makes just two DFS traversals of ·
G, it runs in O(n+m) time.
Computing all Connected Components
When a graph is not connected, the next goal we may have is to identify all of the connected components of an undirected graph, or the strongly connected compo-
nents of a directed graph. We begin by discussing the undirected case.
If an initial call to DFS fails to reach all vertices of a graph, we can restart a new call to DFS at one of those unvisited vertices. An implementation of such a comprehensive DFS all method is given in Code Fragment 14.7.
```
1
def DFS complete(g):
2
···Perform DFS for entire graph and return forest as a dictionary.
3
4
Result maps each vertex v to the edge that was used to discover it.
5
(Vertices that are roots of a DFS tree are mapped to None.)
6
···
7
forest = { }
8
for u in g.vertices():
9
```

Performance of the Edge List Structure

The performance of an edge list structure in ful·lling the graph ADT is summarized in Table 14.2. We begin by discussing the space usage, which is O(n + m) for representing a graph with n vertices and m edges. Each individual vertex or edge instance uses O(1) space, and the additional lists V and E use space proportional to their number of entries.

In terms of running time, the edge list structure does as well as one could hope in terms of reporting the number of vertices or edges, or in producing an iteration of those vertices or edges. By querying the respective list V or E, the vertex count and edge count methods run in O(1) time, and by iterating through the appropriate list, the methods vertices and edges run respectively in O(n) and O(m) time.

The most signi·cant limitations of an edge list structure, especially when compared to the other graph representations, are the O(m) running times of methods get edge(u,v), degree(v), and incident edges(v). The problem is that with all edges of the graph in an unordered list E, the only way to answer those queries is through an exhaustive inspection of all edges. The other data structures introduced in this section will implement these methods more ef·ciently.

Finally, we consider the methods that update the graph. It is easy to add a new vertex or a new edge to the graph in O(1) time. For example, a new edge can be added to the graph by creating an Edge instance storing the given element as data, adding that instance to the positional list E, and recording its resulting Position within E as an attribute of the edge. That stored position can later be used to locate and remove this edge from E in O(1) time, and thus implement the method remove edge(e)

It is worth discussing why the remove vertex(v) method has a running time of O(m). As stated in the graph ADT, when a vertex v is removed from the graph, all edges incident to v must also be removed (otherwise, we would have a contradiction of edges that refer to vertices that are not part of the graph). To locate the incident edges to the vertex, we must examine all edges of E.

Operation	Running Time
vertex count(), edge count()	O(1)
vertices()	O(n)
edges()	O(m)
get edge(u,v), degree(v), incident edges(v)	O(m)
insert vertex(x), insert edge(u,v,x), remove edge(e)	O(1)
remove vertex(v)	O(m)

Table 14.2: Running times of the methods of a graph implemented with the edge list structure. The space used is O(n+m), where n is the number of vertices and m is the number of edges.

DFW
MIA
ORD
JFK
BOS
SFO
LAX
DFW
SFO
MIA
ORD
JFK
BOS
LAX
(a)
(b)
JFK
BOS
LAX
DFW
ORD
MIA
SFO
DFW
MIA
ORD
JFK
BOS
SFO
LAX
(c)
(d)
Figure 14.3: Examples of reachability in a directed graph: (a) a directed path from BOS to LAX is highlighted; (b) a directed cycle (ORD, MIA, DFW, LAX, ORD) is highlighted; its vertices induce a strongly connected subgraph; (c) the subgraph of the vertices and edges reachable from ORD is highlighted; (d) the removal of the dashed edges results in an acyclic directed graph.

Example 14.7: Perhaps the most talked about graph today is the Internet, which can be viewed as a graph whose vertices are computers and whose (undirected) edges are communication connections between pairs of computers on the Internet. The computers and the connections between them in a single domain, like wiley.com, form a subgraph of the Internet. If this subgraph is connected, then two users on computers in this domain can send email to one another without having their information packets ever leave their domain. Suppose the edges of this subgraph form a spanning tree. This implies that, if even a single connection goes down (for example, because someone pulls a communication cable out of the back of a computer in this domain), then this subgraph will no longer be connected.